

Ejercicio - Grupo 1

Contextualización

La librería "Escribiendo Sueños" ha tenido un gran auge en los últimos años, expandiendo su catálogo a miles de títulos de distintos géneros y autores. Sin embargo, su sistema de gestión sigue siendo una hoja de Excel compartida por WhatsApp. Los encargados pierden horas buscando libros, actualizando datos y coordinando con el equipo quién agregó qué.

Por esta razón, Escribiendo Sueños ha contratado sus servicios de desarrollo para que elaboren un sistema interno que le permita al equipo de la librería gestionar su catálogo de libros y autores de forma centralizada: ver, agregar, editar y eliminar registros desde una interfaz moderna.

Para ayudarle en su tarea, les entregan una API (<https://6a05cb38aa826ca75c0a9bc7.mockapi.io/bookstore-api/v1>) con los siguientes endpoints.

- /books
- /authors

Requerimientos

Requerimientos

R1 — Setup Crear el proyecto con routing y agregar Angular Material.

R2 — Estructura de carpetas Respetar la separación en core/, shared/ y features/.

R3 — Modelos Crear una interfaz TypeScript por recurso con todos los campos que devuelve la API.

R4 — Servicios Crear un servicio por recurso con los cinco métodos: getAll, getByid, create, update, delete.

R5 — Header Componente compartido con MatToolbar que muestre el nombre de la app y links a redes sociales con Font Awesome.

R6 — Navbar Componente compartido con links de navegación a **Libros** y **Autores**. El link activo debe tener estilo visual diferente con routerLinkActive.

R7 — Footer Componente compartido visible en todas las pantallas con el año actual y el nombre de la app.

R8 — Lista de libros Página que consume el servicio y muestra cada libro en una MatCard. Tiene tres acciones por card: ver detalle, editar y eliminar. Tiene un botón "Nuevo libro" que abre el formulario. Muestra un MatProgressSpinner mientras carga.

R9 — Detalle de libro Página que lee el :id de la URL y muestra todos los datos del libro. Tiene un botón para volver a la lista.

R10 — Formulario de libro (MatDialog) Dialog con Reactive Forms para crear y editar. El mismo dialog sirve para los dos casos — si recibe un libro como data es edición, si recibe null es creación. Validaciones requeridas en todos los campos. Muestra MatSnackBar al guardar o al fallar.

R11 — Confirmación de eliminación (MatDialog) Dialog que muestra el nombre del libro a eliminar y pide confirmación antes de ejecutar el DELETE. Muestra MatSnackBar al eliminar o al fallar.

R12 — Lista de autores Igual que la lista de libros pero para autores.

R13 — Detalle de autor Igual que el detalle de libro pero para autores.

R14 — Formulario de autor (MatDialog) Igual que el formulario de libro pero para autores.

R15 — Confirmación de eliminación de autor (MatDialog) Igual que el dialog de confirmación pero para autores.

R16 — Routing Cada feature tiene su propio archivo de rutas con lista y detalle. Ambas features se cargan con lazy loading desde app.routes.ts. La ruta raíz redirige a la lista de libros.

R17 — Signals Usar signal() para manejar loading en todos los componentes. Mostrar MatSnackBar en caso de error en cualquier petición HTTP.